

Qualité de Développement

Développement des composants logiciels — SOLID, patterns, conventions, versionning

Ce rapport démontre l'application des principes SOLID, l'utilisation de design patterns adaptés, le respect des bonnes pratiques Clean Code, la stratégie de branches Git et la mise en œuvre des revues de code au sein du projet CareerAI.

Projet	CareerAI — Plateforme SaaS de recherche d'emploi
Stack	Next.js 14 · Supabase · Groq IA · Resend
URL	https://www.careerai.live
GitHub	github.com/ARTHURNJOUONANG/Projet_Chat_board

1. Principes SOLID appliqués

Les principes SOLID guident la conception des composants logiciels de CareerAI pour garantir maintenabilité, évolutivité et testabilité.

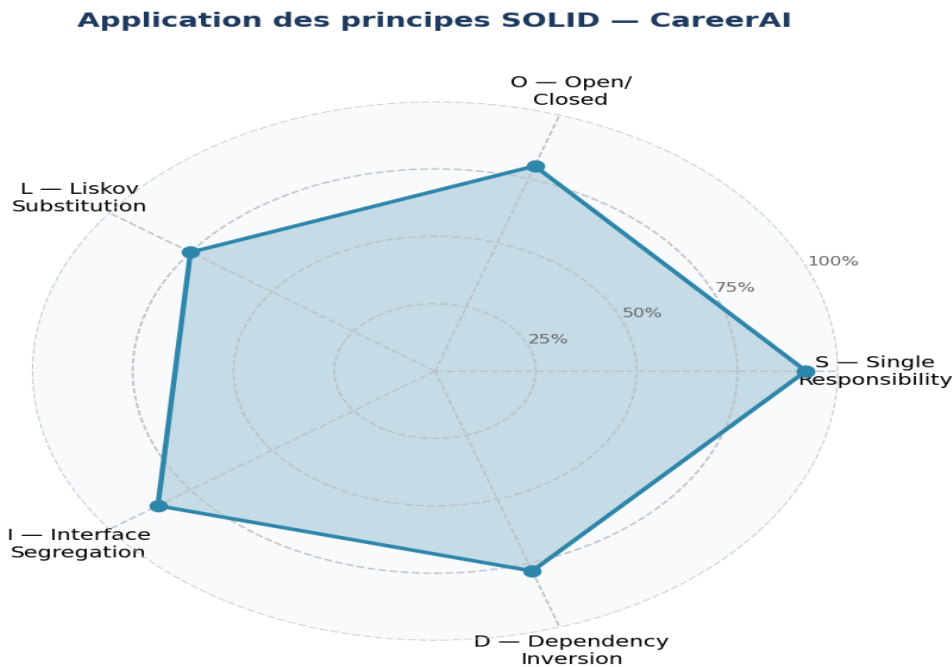


Figure 1 — Application des principes SOLID dans CareerAI

Principe	Application dans CareerAI	Fichier clé
S — Single Responsibility	CampaignService.js gère uniquement la logique des campagnes. Les routes API se limitent à HTTP + auth.	backend/services/CampaignService.js
O — Open/Closed	Nouvelles sources d'offres ajoutées via fonctions fetchJobsFrom* sans modifier le cœur du matching.	backend/services/ (fetchJobsFromAdzuna, etc.)
L — Liskov Substitution	Les services sont interchangeable derrière des interfaces stables (runCampaignDay).	CampaignService, AIService
I — Interface Segregation	API découpée par domaine (/api/campaigns, /api/recruiter) pour éviter les dépendances croisées.	app/api/campaigns/, app/api/recruiter/
D — Dependency Inversion	Les routes reçoivent le client supabase en paramètre ; les services ne dépendent pas de React.	createRouteHandlerClient({ cookies })

2. Design Patterns utilisés

Pattern	Usage	Localisation
Service Layer	Centralise la logique métier, évite la duplication dans les routes.	backend/services/*.js
Repository implicite	Accès Supabase centralisé dans les services (pas d'accès direct à supabase).	backend/services/, backend/lib/
Context (React)	Partage d'état global : auth, toasts, internationalisation	frontend/contexts/ (AuthContext, ToastContext, L

Route Handler	Pattern Next.js App Router — exports GET, POST, PATCH, DELETE, PUT, etc.	app/api/*route.js
Observer (implicite)	Supabase Realtime peut être utilisé pour écouter les changements de données	Supabase Realtime

3. Conventions de code (Clean Code)

Frontend

Convention	Détail
use client	Marqueur obligatoire pour composants interactifs React
Styles	Tailwind CSS — classes utilitaires cohérentes, pas de CSS inline
Appels API	Toujours via frontend/lib/api.js avec credentials: include
Traductions	frontend/lib/translations.js — FR/EN unifié
Nommage	PascalCase pour composants, camelCase pour fonctions et variables

Backend / API

Convention	Détail
Routes	app/api/**/route.js — exports nommés GET, POST, PATCH, DELETE
Session	createRouteHandlerClient({ cookies }) — jamais de service role côté client
Secrets	Uniquement process.env.* côté serveur — aucune clé dans le code
Réponses	NextResponse.json({ error: "...", { status }}) normalisé
Migrations	Idempotentes dans supabase-migrations/ — ordonnées par date

4. Gestion de version et stratégie de branches

Stratégie de branches Git — CareerAI

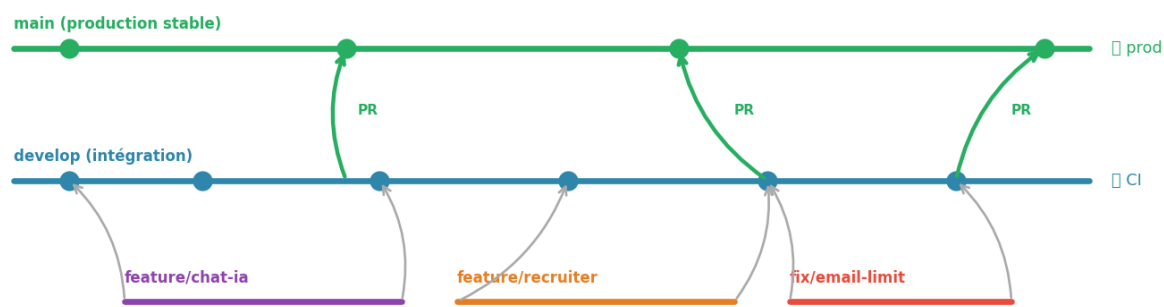


Figure 2 — Stratégie de branches Git Flow

Branche	Usage	Workflow CI
main	Production stable (careerai.live)	Build Android (.github/workflows/build-android.yml)
develop	Intégration continue des features	Tests automatiques + build
feature/*	Nouvelles fonctionnalités isolées	Tests unitaires avant merge
fix/*	Corrections de bugs ciblées	Merge rapide après validation

5. Dette technique connue

Sujet	Statut	Action prévue
Couverture tests partielle	En cours	Extension progressive — voir strategie-tests
OpenAPI partielle	■■ En cours	Routes principales documentées, extension progressive
Cache distribué	■ Roadmap	Redis ou Upstash si volume augmente
E2E tests	■ Roadmap	Playwright sur parcours auth et campagne